PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2026

UE24CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

PESU-Sync Campus Events and Capstone Teaming

Submitted by:

PES1UG24AM184	Parth Pandurang Kale	5 D
PES1UG24AM186	P Manas Siddhardha	5 D

Class of Prof: Prof.Raghu BA
Lab faculty :Devna Sumesh

Abstract

PESU-Sync is a campus platform for forming teams, managing event participation and reviewing capstone proposals. Students maintain skill profiles, find recruiting teams and submit join requests. Faculty manage events and review proposals, while administrators oversee user accounts and audit activity.

PostgreSQL stores authoritative application records and enforces relationships and workflow rules. Qdrant stores embeddings for semantic matching and search. The application uses Next.js to coordinate both databases and display role-specific dashboards. This report presents the schema, representative database operations and student reports based on the supplied demo records.

The earlier verification run passed 38 supplied tests. Additional checks found gaps in deadline enforcement, inactive student admission, concurrent membership changes and capstone visibility. The evaluation section records these limits alongside the implemented features.

Table of Contents

Sl. No	Topic	Page No.
1	Introduction	4
2	User Requirements Specification	5
3	ER Model	6
4	Relational Model	7
5	SQL DDL Statements	8
6	SQL DML Statements	9
7	NoSQL and Vector Database Commands	10
8	Results	11
9	Next.js Project Tree Structure	12
10	Application Code and Demo	13
11	Evaluation Rubrics	14

Introduction

Problem Statement

Team formation for campus events and capstones is often coordinated through informal chats and spreadsheets. Students must read scattered postings to find open teams that need their skills. Organisers have limited visibility into pending requests, seat availability and team participation. Faculty also need a consistent way to review capstone pitches and find related proposals.

Project Scope

PESU-Sync brings together student profiles, team formation, event management, project review and reporting. Students can discover teams by skill and meaning, apply for membership and communicate after joining. Faculty can create events, monitor teams and review capstones. Administrators can moderate users and review changes recorded by database triggers.

Project Contribution

The project combines a structured recruitment workflow with skill-based and semantic matching. A recommendation uses cosine similarity together with overlap between the student's recorded skills and the team's requirements. Capstone discovery supports keyword, semantic and hybrid retrieval. These features address two specific tasks: finding compatible teammates and spotting related capstone ideas before review.

User Requirements Specification

Requirements by Role

Students can register, manage skills and past projects, search for teams, apply for membership and submit capstone proposals. Team leaders can review requests, accept applicants subject to capacity, lock teams and message members. Faculty can manage events and review pitches. Administrators can manage accounts, view aggregate reports and inspect the audit trail.

Technology Stack

PostgreSQL 16 stores the transactional data. Foreign keys connect users, skills, events and teams, while constraints, triggers and procedures enforce rules at write time. SQL views and aggregates provide dashboard reports. Next.js 15, React and TypeScript deliver the browser interface and API routes, with node-postgres for parameterized SQL and connection pooling.

Choice of Vector Database

Qdrant 1.15 stores 384-dimensional vectors for student profiles, team listings and capstone pitches. The local MiniLM model embeds descriptions so a query can match related ideas with different wording. Qdrant ranks by cosine similarity and filters by payload metadata. The API resolves candidate IDs in PostgreSQL to retrieve authoritative rows and apply access rules.

Authentication

The application uses bcrypt for local passwords and a signed JWT in an HTTP-only cookie. It checks the current account and role in PostgreSQL on protected requests. PESU Academy sign-in was exercised with a mock service during verification.

ER Model

Entities and Dependent Relations

The main entities are User, Skill, Past Project, Event, Team, Join Request, Capstone Project and Team Message. Audit Log, Campus Event and Campus Event Sync record operational data. User Skill, Team Member and Team Required Skill are identifying associative relations whose composite keys use their parent keys. The diagram shows all 14 PostgreSQL tables.

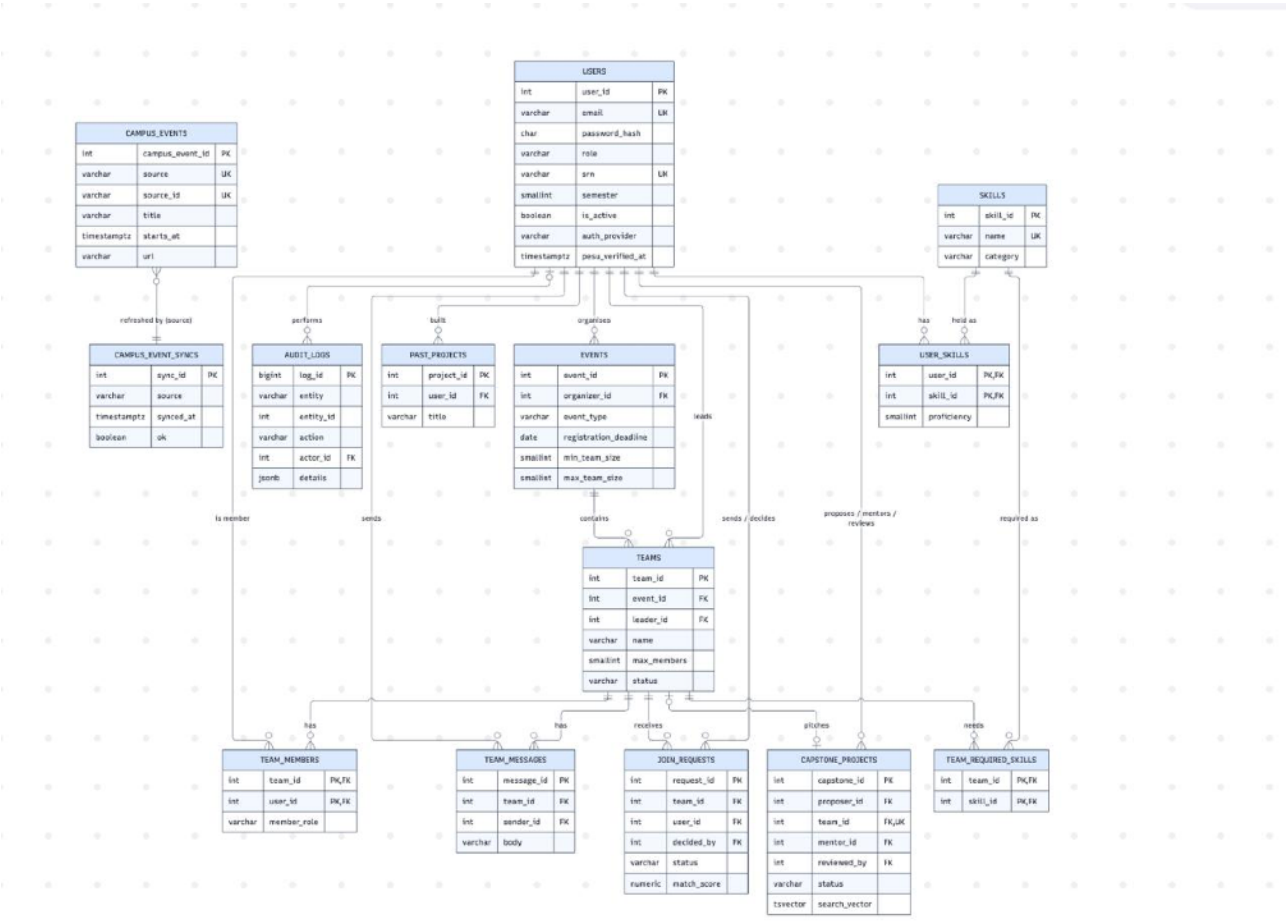


Figure 1. PESU-Sync ER diagram showing all 14 PostgreSQL tables.

Non Simple Attributes

A student's list of skills is multivalued and is stored as separate User Skill rows. Proficiency belongs to each student–skill pair. A team's list of members is represented by Team Member rows. Member counts and open slots are calculated from those rows and team capacity.

Relational Model

Relational Schema and Keys

Figure 1 shows the core foreign-key paths. The relation map below adds the remaining tables. PK identifies each row and FK names a reference to another relation. A pair in parentheses is a composite key.

Relation	PK	FK targets or uniqueness
users	user_id	Unique email and SRN
skills	skill_id	Unique name
user_skills	(user_id, skill_id)	Users and skills
past_projects	project_id	User
events	event_id	User as organiser
teams	team_id	Event and user as leader
team_members	(team_id, user_id)	Team and user
team_required_skills	(team_id, skill_id)	Team and skill
join_requests	request_id	Team, applicant and decision maker
capstone_projects	capstone_id	Proposer, mentor, reviewer and optional unique team
team_messages	message_id	Team and sender
audit_logs	log_id	Optional user as actor
campus_events	campus_event_id	Unique pair (source, source_id)
campus_event_syncs	sync_id	Refresh history by source

Associative relations are user_skills, team_members and team_required_skills. Campus event rows and sync records share a source label. The schema defines no foreign key between those two tables.

SQL DDL Statements

Table and Constraint Definitions

The schema scripts create the tables, account changes, PL/pgSQL routines, reporting views, full-text search objects and campus event cache. The example below shows the key and range constraints for the student-to-skill link.

```
CREATE TABLE user_skills (
    user_id INT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    skill_id INT NOT NULL REFERENCES skills(skill_id) ON DELETE RESTRICT,
    proficiency SMALLINT NOT NULL CHECK (proficiency BETWEEN 1 AND 5),
    PRIMARY KEY (user_id, skill_id)
);
```

Indexes and Reporting Objects

A partial unique index allows at most one pending request for a student and team. A GIN index supports keyword search over the stored capstone search vector. The materialized view mv_skill_demand stores per-skill supply and demand counts for the admin dashboard.

Database Workflow Rules

The acceptance procedure updates the request, inserts membership and resolves other requests in the same event. Membership triggers check capacity and update team status. The audit triggers write selected lifecycle changes to audit_logs. Source files are db/schema/01_schema.sql through db/schema/06_campus_events.sql.

SQL DML Statements

Representative Database Operations

The project inserts users, teams, skills and requests through parameterized statements. Updates capture request decisions, profile edits and event changes. Membership removal deletes a junction row after checking the team state. A team acceptance operation is coordinated by a stored procedure.

```
INSERT INTO user_skills (user_id, skill_id, proficiency)
VALUES ($1, $2, $3);
CALL sp_accept_join_request($1, $2);

SELECT u.department, u.semester, COUNT(*) AS student_count
FROM users AS u
WHERE u.role = 'STUDENT' AND u.is_active
GROUP BY u.department, u.semester
ORDER BY u.department, u.semester;
```

Joins and Correlated Queries

```
SELECT s.category, COUNT(DISTINCT u.user_id) AS students
FROM users AS u
JOIN user_skills AS us ON us.user_id = u.user_id
JOIN skills AS s ON s.skill_id = us.skill_id
WHERE u.role = 'STUDENT' AND u.is_active
GROUP BY s.category
ORDER BY students DESC, s.category;
```

The department participation report uses a correlated EXISTS condition to count students with at least one membership. This avoids double counting when a student joins teams in different events. These aggregate queries illustrate the calculations used in the results section.

NoSQL and Vector Database Commands

Collection Operations

The application creates student_profiles, team_listings and capstone_pitches with vector size 384 and cosine distance. Point IDs match PostgreSQL primary keys. Payloads include fields such as department, event ID, status or domain, depending on the collection.

```
npm run vector:sync
```

```
await client.createCollection(name, {
  vectors: { size: 384, distance: 'Cosine' }
});
await client.upsert(name, { wait: true, points });
await client.query(name, { query: vector, limit: 20, filter });
await client.delete(name, { wait: true, points: staleIds });
```

The examples show the client operations used in src/lib/vector.ts. Upsert inserts a new point or replaces an existing point after its text changes. Query returns ranked candidates subject to filters. Targeted synchronization removes points that are no longer eligible, such as an inactive student or disbanded team.

SQL and Vector Integration

PostgreSQL remains the source for transactional records. A vector write can fail after the SQL transaction commits, so results can temporarily lag. The project can rebuild embeddings using the synchronization script.

Results

The seed dataset contains 25 students. CSE and CSE (AI & ML) each account for 12 students, and ECE accounts for one. Semester 5 contains 12 students and semester 7 contains 13. Python is the most common recorded skill, held by 11 students.

Department	Semester	Students	Share of students
CSE	7	12	48.0%
CSE (AI & ML)	5	12	48.0%
ECE	7	1	4.0%

Figure 2. Department and semester report calculated from the supplied demo data.

Skill category	Students	Student share	Assignments	Avg proficiency / 5
LANGUAGE	22	88%	28	4.04
FRAMEWORK	9	36%	15	3.93
DATABASE	9	36%	14	3.79
AI ML	8	32%	16	3.81
DESIGN	3	12%	6	3.83
DEVOPS	7	28%	12	3.67
DOMAIN	9	36%	13	3.77
SOFT SKILL	3	12%	6	4.33

Figure 3. Skill category counts and mean proficiency in the demo data.

The skill table counts distinct students within each category and 110 individual skill assignments across categories. A student can appear in several categories. Mean proficiency is calculated over the assignments in each category.

Next.js Project Tree Structure

```
pesu-sync/
├── db/
│   ├── schema/            Tables, functions, triggers and views
│   ├── queries/           Representative SQL statements
│   └── seed/              Demo data
├── src/
│   ├── app/
│   │   ├── api/           Next.js Route Handlers
│   │   ├── dashboard/     Role-specific dashboard
│   │   └── ...            Search, events and team pages
│   ├── components/       Shared interface components
│   └── lib/
│       ├── db.ts          PostgreSQL pool and transactions
│       ├── embeddings.ts  Local MiniLM embeddings
│       └── vector.ts      Qdrant setup, sync and queries
├── scripts/               Setup and data synchronization
├── tests/                 Database, vector and API tests
├── docs/                  Technical notes and demo screenshots
└── docker-compose.yml     PostgreSQL and Qdrant services
```

Request Flow

The browser sends a request to a Next.js Route Handler. The handler authenticates the user and checks their role. It queries PostgreSQL or performs a database transaction as needed. A semantic request first embeds the query text and searches Qdrant, then uses the returned IDs to fetch the corresponding SQL records.

Verification

The earlier verification run passed all 38 supplied tests: 23 database, 4 vector and 11 API. It also exercised the browser search and dashboard views. Four targeted checks exposed defects beyond the supplied suite.

Application Code and Demo

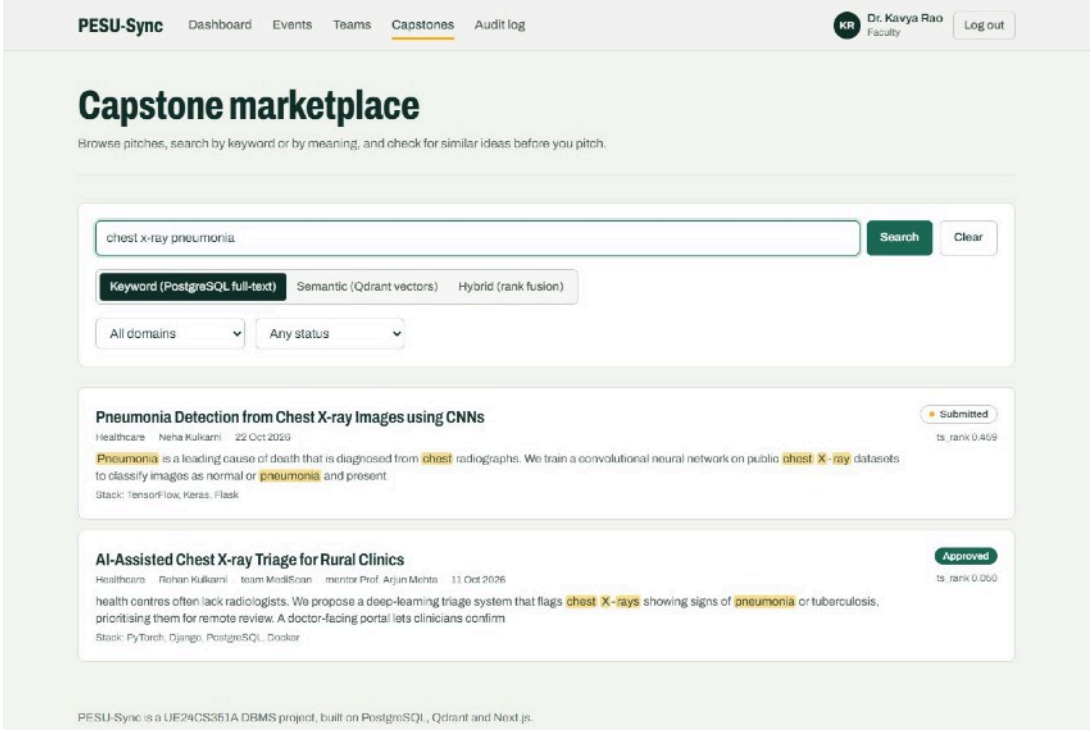


Figure 4. Capstone keyword search in the project demo.

Application Code Sample

The vector helper sends an embedding and optional payload filter to Qdrant. The API uses the returned point IDs to retrieve SQL records.

```
243   const result = await qdrant().query(COLLECTIONS[key], {
244     query: vector,
245     limit: options.limit ?? 10,
246     // eslint-disable-next-line @typescript-eslint/no-explicit-any
247     filter: options.filter as any,
248     with_payload: false,
249   });
250   return result.points.map((p) => ({ id: Number(p.id), score: p.score }));
```

Figure 5. Source excerpt from src/lib/vector.ts.

Demo Workflow

Create a student profile, search for a recruiting team, submit a request and accept it as the team leader. Open the admin dashboard to show department participation and skill demand. Use capstone search to demonstrate keyword and semantic ranking.

Evaluation Rubrics

#	Criteria	Distribution	Status
1	Database Design	ER diagram , relational schema with keys, relationships, and constraints	Implemented
2	Database Creation	SQL database implementation - DDL with constraints , NoSQL/Vector database creation , Technology justification	Implemented
3	SQL Operations and Queries	DML operations , Nested/correlated/Recursive SQL queries, Join SQL queries	Implemented
4	Advanced Database Concepts	Advanced database concepts (Procedure/function, trigger, CTE) , NoSQL/Vector feature RUD	Implemented
5	Web Application and Database Integration	Next.js interface , Backend/API and database integration	Implemented
6	Authentication and Authorization	Authentication implementation (front end and backend) and minimum two roles and role-based access control in database	Implemented
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub , Originality and individual contribution	Implemented

Open Findings

The earlier verification found that a team's optional deadline was not enforced, a pending request from a deactivated student could be accepted, concurrent SQL writes could admit one student into two teams for an event, and related-pitch suggestions could reveal rejected pitch metadata.